

Le cryptosystème RSA

22 septembre 2026 – B. COLOMBEL

Le compte-rendu de ce TP sera fait sous la forme d'un notebook Jupyter. Il sera à déposer sous le nom

Nom_Prenom-TP3.ipynb

dans le dossier relatif à ce TP dans AMETICE.

Sommaire

1	Calcul de puissances modulaires	1
2	Rappels de cours	2
3	Mise en œuvre sur un exemple simple	2
3.1	Génération des clés	2
3.2	Chiffrement / Déchiffrement	4
4	Un exemple grandeur nature	4
4.1	Obtenir de grands nombres premiers	4
4.2	Calcul de $\varphi(n)$	5
4.3	choix des clés	6
4.4	Chiffage et déchiffage d'un message	6
5	Utilisation et attaque du RSA	7
5.1	Algorithme de Fermat	7
5.2	Attaque de Wiener	8

Dans tout le TP,

- n désigne la partie nommée modulus d'une clé RSA ; c'est le produit de deux nombres premiers distincts p et q (gardés secrets), tandis que n est rendu public ;
- les exposants de chiffrement (public) et déchiffrement (privé) sont notés e et d .

1 Calcul de puissances modulaires

Le chiffrement comme le déchiffrement par RSA reposent sur le calcul de puissances modulaires, c'est-à-dire le calcul de :

$$a^r \bmod m$$

Le calcul direct consistant à déterminer a^r puis sa congruence modulo m ne peut être entrepris pour des grandes valeurs de a et r . En cours, nous avons évoqué l'algorithme des carrés pour résoudre ces problèmes afin d'effectuer ce calcul.

Si on récapitule le fonctionnement de l'algorithme, pour calculer $a^r \bmod m$, on suit les étapes suivantes :

- On initialise $p = 1$
- à chaque étape
 - si r impair, on remplace p par $p \times a$ et r par $\frac{r-1}{2}$
 - si r pair, on remplace juste r par $\frac{r}{2}$
- dans tous les cas, on remplace a par a^2
- tous les calculs sont bien sûr faits modulo m

— l'algorithme s'arrête lorsque $r = 0$
le résultat recherché est donné par la valeur finale de p .

Exercice 1. Écrire une fonction `expo_mod(a, r, m)` en *sagemath* qui permet de calculer le nombre $a^r \bmod m$.

Remarque. On pourra utiliser :

- une boucle `while` puisque l'algorithme tourne tant que r est non nul, et une instruction `if` qui teste si r est pair ou non (on pourra vérifier si son reste modulo 2 est nul ou égal à 1) ;
- intensivement la commande `a % b`, qui donne le reste de la division euclidienne de a par b , pour travailler à chaque fois avec des nombres les plus petits possible.

Exemple d'utilisation.

```
sage : expo_mod(13, 5, 8)
5
```

2 Rappels de cours

Le chiffrement et le déchiffrement d'un message en RSA s'appuie sur les théorèmes :

Théorème 1 (cas particuliers du théorème d'Euler)

- Soient p un entier premier et x un entier premier avec p . Alors $x^{(p-1)} \equiv 1 \bmod p$.
- Soient p, q deux entiers premiers, $n = p \times q$. Soit x un entier premier avec n , alors $x^{(p-1)(q-1)} \equiv 1 \bmod n$.

Théorème 2 (Rivest-Shamir-Adleman)

Soient p et q deux entiers premiers, et $n = p \times q$. Soient d et e inverses modulo $(p-1)(q-1)$, c'est-à-dire $e \times d = 1 + k(p-1)(q-1)$ pour $k \in \mathbb{Z}$. Alors pour tout $x \in \mathbb{Z}$, on a

$$x^{ed} \equiv x^{1+k(p-1)(q-1)} \equiv x \left(x^{(p-1)(q-1)} \right)^k \equiv x \bmod n$$

Méthode :

- Soient $p \neq q$ deux nombres premiers. On pose $n = p \times q$.
Pour n très grand, il est très difficile de déterminer p et q à partir de n .
- Soient d et e inverses modulo $(p-1)(q-1)$. On est dans les conditions d'application du théorème RSA.
- Il est possible de construire facilement de tels entiers d et e quand on connaît la décomposition $n = p \times q$.
 - On choisit un nombre e non divisible par $p-1$ ni $q-1$
 - On calcule d inverse de e modulo $(p-1)(q-1)$ par l'algorithme d'Euclide étendu.

Chiffrement et déchiffrement : Bob veut envoyer un message M à Alice...

- *Règle de codage* : $x \mapsto x^e \bmod n$. Bob calcule, au moyen de la clé publique d'Alice (e, n) , le nombre $C = M^e \bmod n$.
- *Règle de décodage* : $x \mapsto x^d \bmod n$. Alice calcule, au moyen de sa clé privée (d, n) , le nombre $C^d \bmod n$, et retrouve bien M par le théorème RSA.

3 Mise en œuvre sur un exemple simple

3.1 Génération des clés

On prend $p = 47$ et $q = 59$.

```
sage: p = 47 ; q = 59
```

On vérifie que p et q sont bien premiers.

```
sage: p.is_prime()
True
sage: q.is_prime()
True
```

On peut alors calculer, $n = p \cdot q$ et la fonction indicatrice d'Euler $\varphi(n)$

```
sage: n = p * q
sage: phi = euler_phi(n)
sage: n, phi
(2773, 2668)
```

Il convient ensuite de trouver un entier $e \in \mathbb{Z}$ tel que $\text{pgcd}(e, \varphi(n)) = 1$.

Il s'agit de prendre un nombre n'ayant aucun facteur premier commun avec $\varphi(n)$. Pour cela, on peut s'aider par exemple de la factorisation de φ .

```
sage: factor(phi)
2^2 * 23 * 29
```

On peut donc prendre $e = 17$ par exemple. En pratique, pour une véritable implémentation de RSA, il vaut mieux éviter de choisir e trop petit.

```
e = 17
```

Ensuite, nous pouvons déterminer la clé privée d de déchiffrement. Il s'agit de trouver un entier $d \in \mathbb{Z}$ tel que :

$$d \cdot e \equiv 1 \pmod{\varphi(n)}$$

Comme e et $\varphi(n)$ sont premiers entre eux, e est inversible modulo $\varphi(n)$.

Autrement dit il existe un entier $d \in \mathbb{Z}$ tel que $d \cdot e \equiv 1 \pmod{\varphi(n)}$ comme voulu.

En pratique, on utilise l'algorithme d'Euclide étendu : il fournit deux entiers d et s tels que

$$de + s\varphi(n) = 1$$

On a ainsi

$$d \cdot e \equiv 1 \pmod{\varphi(n)}$$

Si jamais d est négatif, il suffit de le remplacer par son représentant canonique dans $\mathbb{Z}/\varphi(n)\mathbb{Z}$ de manière à avoir un entier naturel.

Le résultat de l'algorithme d'Euclide étendu entre deux entiers a et b s'obtient par la commande `xgcd(a, b)`. Consulter l'aide de Sage pour comprendre à quoi correspondent les trois valeurs retournées par cette fonction.

```
sage: xgcd(e, phi)
(1, 157, -1)
```

On peut donc prendre $d = 157$. On vérifie :

```
sage: d = 157
sage: (d * e) % phi
1
```

Nous avons ainsi les clés :

PUBLIC KEY

$P_{key} = (\text{RSA}, n = 2773, e = 17)$

SECRET KEY

$S_{key} = (\text{RSA}, n = 2773, d = 157)$

3.2 Chiffrement / Déchiffrement

Le message à chiffrer est ITS ALL GREEK TO ME.

On code l'espace par 00, puis les lettres de A à Z par 01 à 26. Le message devient :

$$M = 0920\ 1900\ 0112\ 1200\ 0718\ 0505\ 1100\ 2015\ 0013\ 0500.$$

Pour ne pas manipuler les nombres trop grands, et parce que l'on doit chiffrer des valeurs inférieures à $n = 2773$, on applique le codage RSA à chaque paquet de 4 chiffres. Le premier paquet $M_1 = 0920$ est codé avec la clé publique :

Le calcul de puissance modulaire est effectué par l'algorithme des carrés.

```
sage: expo_mod(920, e, n)
948
```

On vérifie :

```
sage: expo_mod(948, d, n)
920
```

4 Un exemple grandeur nature

4.1 Obtenir de grands nombres premiers

Théorème 3 (Théorème des nombres premiers)

Pour tout $x > 0$, on note $\pi(x)$ le nombre de nombres premiers inférieurs ou égaux à x . Alors, $\pi(x)$ est équivalente, lorsque x tend vers $+\infty$, au quotient de x par son logarithme népérien :

$$\pi(x) \sim \frac{x}{\ln(x)}$$

Cela signifie que

$$\lim_{x \rightarrow +\infty} \pi(x) \frac{\ln(x)}{x} = 1$$

et donc que, pour x « assez grand », le nombre de nombres premiers inférieurs à x peut être approximé par $\frac{x}{\ln(x)}$.

Une interprétation de ce théorème en termes probabilistes est la suivante : la probabilité pour qu'un entier naturel d'au plus k chiffres et choisi au hasard soit premier est de l'ordre de $\frac{1}{\ln(10^k)}$.

Pour fabriquer au hasard de grands nombres premiers, on peut donc procéder ainsi : on tire au hasard un entier et on teste s'il est premier ou non. S'il ne l'est pas, on en essaye un autre, etc. jusqu'à en trouver un.

Le résultat précédent assure que cette méthode va aboutir après un nombre suffisant et raisonnable de lancers avec une probabilité élevée.

Supposons que l'on cherche à déterminer un nombre premier à 310 chiffres. On note $q = 1 - \frac{1}{\ln 10^k}$ la probabilité pour qu'un entier naturel d'au plus k chiffres et choisi au hasard ne soit pas premier. La probabilité d'avoir n échecs successifs est égale à q^n .

Elle est inférieure à 5% dès lors que :

$$n \geq \frac{\ln 0,05}{\ln q}$$

```
sage: k = 310
sage: q = 1-1/ln(10**k)
sage: ln(0.05) / ln(q).n()
2136.85961134156
```

Ainsi, pour $k = 310$, nous avons 95 % de chance de trouver un nombre premier en testant 2 136 nombres entiers.

Remarque. Malheureusement, les tests de primalité déterministes sont lents. Dans la pratique on utilise des algorithmes qui testent si le nombre est « *pseudo-premier* », c'est-à-dire, premier avec une grande probabilité (méthode `is_pseudoprime()`) puis lorsqu'un candidat a réussi ce test, à confirmer sa primalité par un test déterministe (méthode `is_prime()`)

```
sage: p = ZZ.random_element(10^312) # On choisit au hasard un nombre entier inferieur a 10^312
sage: N = 1 # On initialise le compteur pour le nombre de tests
sage: while (len(p.digits()) < 310) or (not p.is_pseudoprime()):
....:     p = ZZ.random_element(10^312)
....:     N += 1
```

La boucle `while` fait recommencer le test dès lors que le nombre comporte moins de 310 chiffres ou qu'il n'est pas premier.

La valeur finale de N retourne le nombre de tests effectués.

```
sage: N
883
```

Nombre de chiffres de p :

```
sage: len(p.digits())
310
```

On peut aussi afficher le nombre p .

```
sage: p
5171580075478931594509360412348407766342210294522059751636118270640506099833200190938948490555
2094870375598598430016472322509555705267350496700893495163926881229099888666976749337348684008
2512338933731996459237216480672992658123322297095622665270892135162788565075722225050864864633
7166668774289018643141435141
```

Il reste à confirmer sa primalité par un test déterministe. La commande `%time` permet d'afficher le temps utilisé pour exécuter une commande.

```
sage: %time p.is_prime()
CPU times: user 2.43 s, sys: 20.4 ms, total: 2.45 s
Wall time: 2.72 s
True
```

Exercice 2. De la même manière, déterminer un autre nombre premier q comportant au moins 310 chiffres (et moins que 312). Donner le nombre de chiffres de q . Quelle est la valeur de n correspondant ?

4.2 Calcul de $\varphi(n)$

Vous pouvez tenter la commande

```
sage: euler_phi(n) # Trop long !
```

mais un `Ctrl`+`C` va s'imposer rapidement. C'est trop long !

Le calcul de $\varphi(n)$ par la commande `euler_phi(n)` prend beaucoup de temps. C'est normal car *sagemath* va chercher à factoriser n afin de calculer $\varphi(n)$. Comme n a plus de 610 chiffres, c'est une tâche impossible en un temps raisonnable. La sécurité de RSA réside ici.

Pour calculer $\varphi(n)$, on se rappelle que comme $n = p \cdot q$ avec p et q premiers distincts, on a

$$\varphi(n) = \varphi(p)\varphi(q) = (p-1)(q-1)$$

On demande à *sagemath* de calculer directement ce nombre, qui suppose de connaître les facteurs p et q de n (clé privée).

Exercice 3. Quelle valeur de $\varphi(n)$ trouvez vous ? L'afficher.

4.3 choix des clés

Clé publique. On cherche un entier e tel que :

$$\text{pgcd}(e, \varphi(n)) = 1$$

Là encore, il n'est pas question de factoriser $\varphi(n)$, qui est très grand !

Par contre, la majorité des entiers plus petits que $\varphi(n)$ n'ont pas de facteurs premiers communs avec $\varphi(n)$. On procède donc, encore une fois, en tirant des nombres au hasard, jusqu'à en trouver un qui réalise la condition souhaitée.

```
sage: e = ZZ.random_element(phi)
sage: while gcd(e, phi) != 1 :
....:     e = ZZ.random_element(phi)
....:
```

On vérifie :

```
sage: e.gcd(phi)
1
```

Clé privé. On cherche un entier $d \in \mathbb{N}$ tel que

$$de \equiv 1 \pmod{\varphi(n)}$$

À nouveau, c'est l'algorithme d'Euclide étendu qui permet de le faire.

```
sage: d = xgcd(e, phi)[1]
```

Nous avons ainsi nos deux clés (publique et privée) :

Clé publique : (n, e) **Clé privée :** (n, d)

4.4 Chiffrage et déchiffrage d'un message

On choisit un message numérique m au hasard avec $m < n$.

```
sage: m = ZZ.random_element(n)
sage: m < n # on vérifie que m est bien strictement inférieur à n
True
```

Pour chiffrer le message, tentons un :

```
sage: (m^e)%phi # Trop long !
```

La commande ci-dessus prend beaucoup trop de temps (voire génère une erreur si la puissance est trop grande) : elle fait calculer m^e avant de la réduire modulo n . Ce n'est pas une manière adaptée de calculer des puissances modulo n . Il vaut mieux utiliser l'exponentiation rapide programmée dans la partie 1.

```
sage: %time c = expo_mod(m, e, n)
CPU times: user 14.4 ms, sys: 23 µs, total: 14.4 ms
Wall time: 14.3 ms
```

On déchiffre ensuite le message à l'aide de notre clé privée. Pour cela, on évalue $c^d \pmod{n}$, en utilisant encore l'exponentiation rapide.

```
sage: expo_mod(c, d, n)
```

On peut vérifier que le message initial, et le message chiffré puis déchiffré coïncident.

```
sage: expo_mod(c, d, n) == m
True
```

5 Utilisation et attaque du RSA

On utilise RSA essentiellement pour communiquer de manière confidentielle une clé d'un système à clé secrète.

5.1 Algorithme de Fermat

Supposons que l'entier n soit le produit de deux nombres premiers p et q « proches » (on peut toujours supposer que $p > q$).

On pose :

$$t = \frac{p+q}{2} \quad \text{et} \quad s = \frac{p-q}{2}$$

L'entier s est donc petit (« proche » de zéro).

par ailleurs,

$$\begin{aligned} t^2 - s^2 &= (t-s)(t+s) \\ &= \left(\frac{p+q}{2} - \frac{p-q}{2} \right) \left(\frac{p+q}{2} + \frac{p-q}{2} \right) \\ &= \left(\frac{p+q-p+q}{2} \right) \left(\frac{p+q+p-q}{2} \right) \\ &= \frac{2q}{2} \cdot \frac{2p}{2} \\ &= p \cdot q \\ t^2 - s^2 &= n \end{aligned}$$

Ainsi, comme s est proche de 0, t est légèrement inférieur à la racine de n . On en déduit l'algorithme suivant :

Algorithme de Fermat	
Entrées : un entier n (impair)	
Initialisation : $t \leftarrow$ Partie entière de $\sqrt{n} + 1$; $z \leftarrow t^2 - n$;	
1	début
2	tant que z n'est pas un carré faire
3	$t \leftarrow t + 1$;
4	$z \leftarrow t^2 - n$;
5	fin
6	fin
Sorties : Retourner $p = t - \sqrt{z}, t + \sqrt{z}$	

Exercice 4. Un fichier a été chiffré en utilisant l'AES en mode CBC avec un vecteur d'initialisation nul et une clé de 128 bits. Cette clé K a été elle-même chiffrée avec la clé publique RSA.

$$\begin{aligned} n &= 4840015169768242918240815055699674259180276588222516131662837 \\ e &= 65537 \end{aligned}$$

et on obtient :

$$K^e \pmod{n} = 2336273333675885101548598149697595180856150539608777837370662$$

1. Le modulus n est un produit de deux nombres premiers assez proches l'un de l'autre. Il est possible de le factoriser par la méthode de Fermat. Faites-le.
2. Déduisez-en la clé privée d , puis retrouvez la clé secrète K .

5.2 Attaque de Wiener

M. Wiener a publié en 1990 (Cryptanalysis of Short RSA Secret Exponents, IEEE Transaction on Information Theory, vol 36, no 3, mai 1990) une attaque qui permet de retrouver la clé privée d en connaissant uniquement la clé publique e et n lorsque cette clé privée d est inférieure à la racine quatrième du modulus n .

Sa technique s'appuie sur le développement en fraction continue de e/n . Parmi les réduites obtenues l'une a pour dénominateur l'exposant privé d .

Voici un exemple :

```
#####
# Attaque de Wiener
#####
# la clé publique
sage: n = 7507749913 ; e = 3217382455

#calcul des réduites de la fraction continue associée à e/n
sage: cvgts = continued_fraction(e/n).convergents()

#voici les réduites
sage: cvgts
[0,
 1/2,
 2/5,
 3/7,
 2045/4772,
 6138/14323,
 45011/105033,
 51149/119356,
 3011653/7027681,
 3062802/7147037,
 6074455/14174718,
 9137257/21321755,
 24348969/56818228,
 57835195/134958211,
 82184164/191776439,
 222203523/518511089,
 748794733/1747309706,
 3217382455/7507749913]

# Recherche de la clé privée parmi les dénominateurs des réduites
sage: m = 12345
sage: c = power_mod (m, e, n)
sage: i = 0
sage: while i <= len(cvgts) and power_mod ( c , cvgts[ i ].denominator(), n) != m :
....:     i += 1
....:
sage: i, cvgts[i].denominator()
(3, 7)
# BINGO, on a trouvé la clé secrète (d = 7)
```

Exercice 5. Les cryptogrammes suivants ont été produits avec des clés RSA dont l'exposant de déchiffrement satisfait l'hypothèse de l'attaque de Wiener.

```
# chiffré
mc = 783829455698841046080191161847433696552376297766231340655925365925283689720739921446106628
47836540384572024038300445043766487630243604142062997708360896189100881098719398382003545481258
16096570413755061613485229318236280200904889067968609369568142098062420460396137162769658214493
```

```
917660515828417381339578346
```

```
# modulus n
```

```
n = 7260769079638221953385560157422697983935892589478301226474619224231206476180746968655603279  
89977672186524038499927841509828521020671957849245534810000619316271002615010904204768191588911  
07478865521004842594565621865761568219736813805495036064204106467048980512337940858538750651798  
890768604018328115165005573
```

```
# clé publique
```

```
e = 6771597234057378364716118433029817839390670196399192710802762814105907214814473500441941350  
93898387521607321728974648113803980847882069014783760771076387690676440930247896667015586547005  
21392247661822616688334643664821383484032774980431792606135434693625403907803225216173220349070  
640509336914301659248439067
```

Retrouvez le texte original en utilisant la procédure décrite ci dessous.

Procédure de codage. Le cryptogramme a été obtenu par le codage de petits textes en français transformés en nombres entiers. L'alphabet utilisé pour les textes est constitué des 26 lettres (majuscules et non accentuées) de l'alphabet latin, ainsi que de l'espace. L'alphabet comporte ainsi 27 caractères.

Le codage utilisé pour transformer un texte en un nombre consiste à transformer chaque lettre en un nombre compris entre 10 et 36 et à concaténer les nombres entre eux. Avec ce codage, le texte PAC est transformé en le nombre 261113